



Université de Sherbrooke

SQL

Tables et contraintes simples

SQL_02

Christina KHNAISSER (christina.khnaisser@usherbrooke.ca)

Luc LAVOIE (luc.lavoie@usherbrooke.ca)

Ameni SOUID (ameni.Souid@usherbrooke.ca)

(les auteurs sont cités en ordre alphabétique nominal)

—

Scriptorum/Scriptorum/SQL_02c-Tables-et-cles, version 1.0.0.a, en date du 2026-01-13

— document de travail, ne pas citer —

Sommaire

...

Mise en garde

Le présent document est en cours d'élaboration ; en conséquence, il est incomplet et peut contenir des erreurs.

Historique

diffusion	resp.	description
2026-01-13	CK	Récupération de notes diverses.

Table des matières

Introduction.....	4
1. Présentation.....	4
2. Création de tables.....	4
2.1. Syntaxe (PostgreSQL).....	4
2.2. Exemple — Bibliovik.....	5
3. Retrait de tables.....	6
3.1. Syntaxe (PostgreSQL).....	6
3.2. Exemple — Bibliovik.....	6
4. Modification de tables.....	6
4.1. Syntaxe (PostgreSQL).....	6
4.2. Exemple — Bibliovik.....	8
Conclusion.....	9
Références.....	10
Définitions.....	11

Introduction

Le présent document a pour but de présenter une synthèse du langage de définition des tables et des contraintes de clé qu'on retrouve dans SQL.

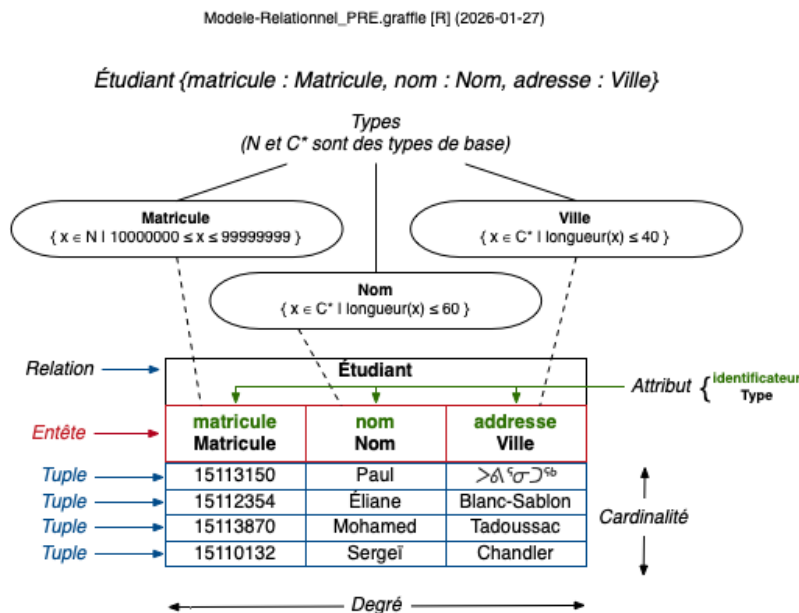
Dans un premier temps, nous présenterons une grammaire simplifiée, sous-ensemble propre de la grammaire de la norme ISO 9075:2016, avec, au besoin, certaines particularités de PostgreSQL.

1. Présentation

Une relation, en tant qu'objet de la théorie relationnelle, est un ensemble de tuples.

En regard de la théorie des types, il s'agit d'un constructeur (de types non scalaires) applicable à un ensemble de tuples de même type.

Une relation est la représentation d'un prédicat logique. qui correspond un ensemble d'observations de même «nature».



En SQL, le constructeur de tables permet de définir une relation.

...

2. Création de tables

La création d'une table dans une base de données.

2.1. Syntaxe (PostgreSQL)

```
CREATE TABLE [ IF NOT EXISTS ] table_name (  
  [{ column_name data_type [ column_constraint [ ... ] ]  
    | table_constraint  
    | LIKE source_table [ like_option ... ] }  
  [, ... ]  
)  
[ INHERITS ( parent_table [, ... ] ) ]
```

where column_constraint is:

```
{ NOT NULL |
  NULL |
  DEFAULT default_expr |
  GENERATED ALWAYS AS IDENTITY }
```

where table_constraint is:

```
CONSTRAINT constraint_name
{ CHECK ( expression ) |

  UNIQUE [ NULLS [ NOT ] DISTINCT ] ( column_name [, ... ] [, column_name WITHOUT
OVERLAPS ] ) index_parameters |

  PRIMARY KEY ( column_name [, ... ] [, column_name WITHOUT OVERLAPS ] )
index_parameters |

  FOREIGN KEY ( column_name [, ... ] ) REFERENCES reftable [ ( refcolumn[, ... ] ) ]
}
[ DEFERRABLE | NOT DEFERRABLE ]
[ INITIALLY DEFERRED | INITIALLY IMMEDIATE ]
[ ENFORCED | NOT ENFORCED ]
```

2.2. Exemple — Bibliovik

La société Bibliovik assure la gestion du réseau des bibliothèques sur le territoire du Nunavik. Elle désire conserver un inventaire des exemplaires de livre qu'elle possède, leur statut, les prêts et les abonnés. Un livre est décrit par un code, un auteur, un titre, un éditeur et une année de parution. Une bibliothèque est décrite par un code et le nom du village où elle se trouve. Chaque exemplaire est décrit par une référence au livre, une référence à la bibliothèque qui le possède, son numéro d'exemplaire, sa date d'acquisition et son statut (disponible, perdu). Finalement, on consigne les prêts (exemplaire, date d'emprunt, date de retour, abonné emprunteur) et les abonnés (numéro d'abonné, nom, prénom, adresse, date de naissance, date d'abonnement).

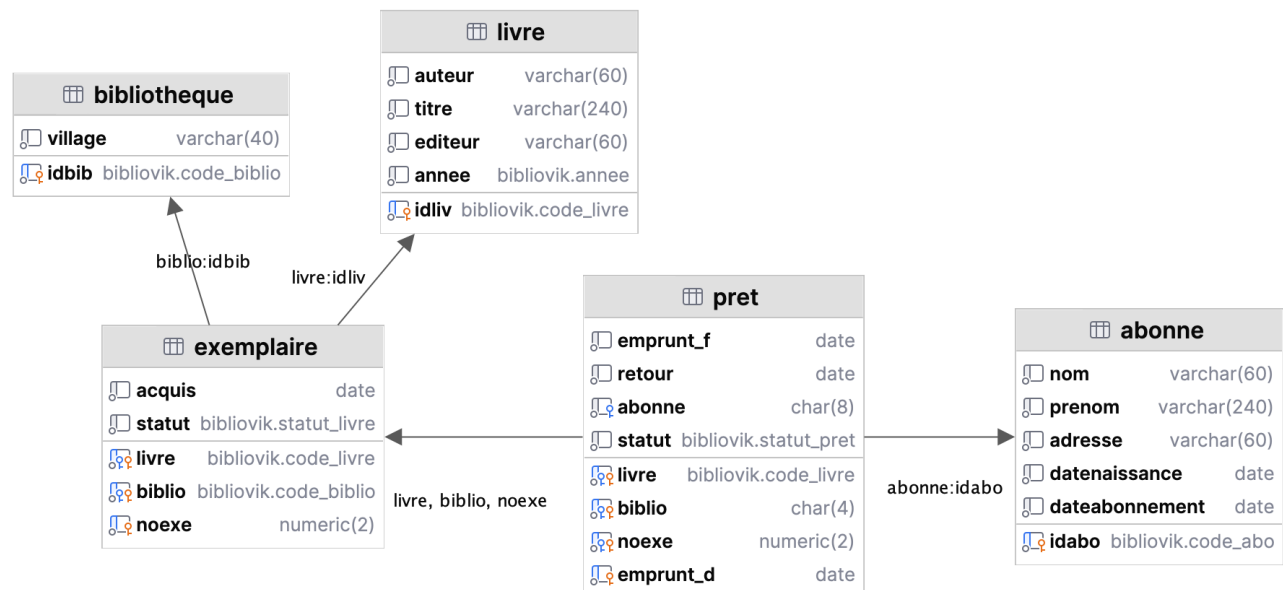


Tableau 1. Dictionnaire des types

Nom	Domaine des valeurs	Description
Code_livre	Chaîne de caractères qui débute par deux lettres en majuscules suivies de huit chiffres.	Un code unique d'un livre attribué en fonction de son sujet.
Annee	Entier	Un entier qui représente une année après 1600.
Statut_livre	Caractère	D = disponible, P = Perdu

3. Retrait de tables

La retrait d'une table d'une base de données.

3.1. Syntaxe (PostgreSQL)

```
DROP TABLE [ IF EXISTS ] name [, ...] [ CASCADE | RESTRICT ]
```

3.2. Exemple — Bibliovik

```
DROP TABLE IF EXISTS Pret;
```

IF EXISTS évite une erreur si la table Pret n'existe pas.

```
DROP TABLE Livre RESTRICT;
```

RESTRICT empêche la suppression de Livre si d'autres objets en dépendent.

```
DROP TABLE Livre CASCADE;
```

CASCADE supprime Livre ainsi que les objets qui dépendent de cette table.

4. Modification de tables

La modification de la structure et les contraintes d'une table dans la base de données.

4.1. Syntaxe (PostgreSQL)

```
ALTER TABLE [ IF EXISTS ] name
  action [, ... ]
```

```
where action is:
  [ RENAME | ADD | DROP | ALTER | SET ]
```

Exemples

```
-- Ajouter une colonne
ALTER TABLE Livre
ADD COLUMN isbn VARCHAR(13);

-- Modifier une colonne
```

```
ALTER TABLE Livre
ALTER COLUMN titre TYPE VARCHAR(300);

-- Supprimer une colonne
ALTER TABLE Livre
DROP COLUMN isbn;

-- Renommer une colonne
ALTER TABLE Livre
RENAME COLUMN editeur TO maison_edition;
```

Renommage

```
RENAME [ COLUMN ] column_name TO new_column_name

RENAME CONSTRAINT constraint_name TO new_constraint_name

RENAME TO new_name
```

Exemples

```
-- Renommer une colonne
ALTER TABLE Livre
RENAME COLUMN editeur TO maison_edition;

-- Renommer une contrainte
ALTER TABLE Livre
RENAME CONSTRAINT Livre_auteur
TO Livre_auteur_check;

-- Renommer une table
ALTER TABLE Bibliotheque
RENAME TO Biblio;
```

Modifications d'une colonne

```
ADD COLUMN column_name data_type [ column_constraint ]

DROP COLUMN column_name [ RESTRICT | CASCADE ]

ALTER COLUMN column_name
[ TYPE data_type |
  SET DEFAULT expression |
  {SET | DROP} SET NOT NULL |
```

Exemple — Ajout d'une colonne

```
ALTER TABLE Livre
ADD COLUMN isbn VARCHAR(13);
```

La colonne isbn est ajoutée à la table Livre.

Modifications d'une contrainte

```
ADD table_constraint
```

```
DROP CONSTRAINT constraint_name  
  [ DEFERRABLE | NOT DEFERRABLE ]  
  [ INITIALLY DEFERRED | INITIALLY IMMEDIATE ]  
  [ ENFORCED | NOT ENFORCED ]  
  
ALTER CONSTRAINT constraint_name
```

4.2. Exemple — Bibliovik

Exemple — Ajout d'une contrainte

```
ALTER TABLE Livre  
ADD CONSTRAINT Livre_titre_editeur_unique  
UNIQUE (titre, editeur);
```

Exemple — Suppression d'une contrainte

```
ALTER TABLE Livre  
DROP CONSTRAINT Livre_editeur;
```

La contrainte Livre_editeur est retirée de la table.

La combinaison titre et editeur doit maintenant être unique.

Conclusion

Références

PG

[fr] <https://docs.postgresql.fr/18/index.html> (2026-01-05)

[en] <https://www.postgresql.org/docs/18/index.html> (2026-01-05)

Définitions

SQL (*Structure Query Language*)

Langage de programmation axiomatique fondé sur un modèle inspiré de la théorie relationnelle proposée par E. F. Codd.

[Normes applicables : ISO 9075:2016, ISO 9075:2023]

Produit le 2026-09-16 16:46:09 UTC



Université de Sherbrooke